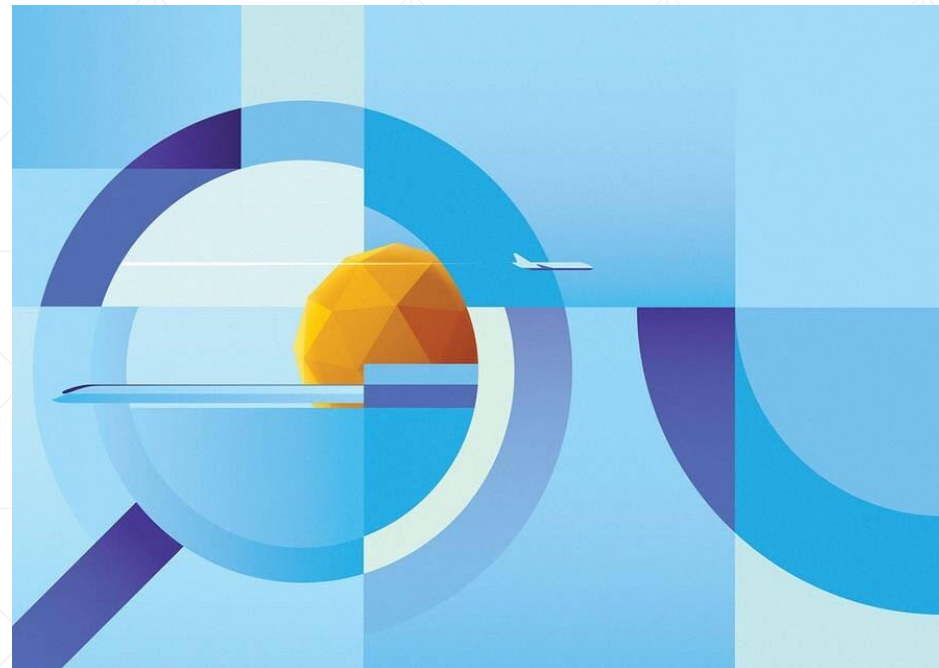




使用Shape绘制矢量图形

北京理工大学计算机学院
金旭亮

概述



JavaFX提供了一组内置的“图形（Shape）”对象可供使用。

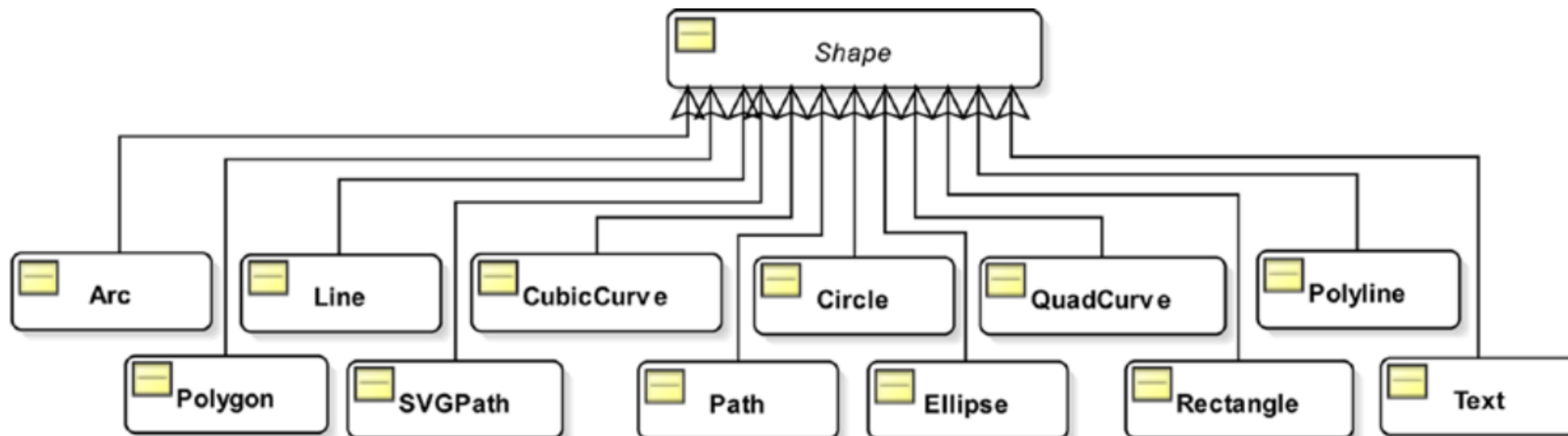


与按钮等UI控件相比，“图形”对象比较轻量级，主要用于矢量绘图，可以实现比较高性能的图形与动画。



图形对象的基类为Shape，它派生自Node，定义了所有图形都共享的特性，比如，`fill`属性定义了图形的填充方式，而`stroke`和`strokeWidth`则用于确定绘制图形的线条和它的宽度。

JavaFX所提供的基本图形组件



图形对象可以加入到场景图中，因为它们都可以被看成是Node类型的实例。

每个Shape都有大小和位置相关的属性，并且不会随着容器尺寸的变化而动态变化，必须写代码手动设置。

Line: 代表一条线段

重要属性:

$(\text{startX}, \text{startY}) - (\text{endX}, \text{endY})$

可以设定它的颜色、粗细和线型。



LineShapes.java

使用Line的示例代码

```
//图形对象的容器，通常选择轻量级的Group
Group root = new Group();

//通过指定起始点和终点坐标确定一条直线
Line line1 = new Line( startX: 20, startY: 50, endX: 400, endY: 50);
//设定使用的线型
line1.setStroke(Color.GREEN);
//指定线条宽度
line1.setStrokeWidth(10);

Line line2 = new Line( startX: 20, startY: 80, endX: 400, endY: 80);
line2.setStroke(Color.BLUE);
line2.setStrokeWidth(10);
line2.setStrokeLineCap(StrokeLineCap.ROUND);

Line line3 = new Line( startX: 20, startY: 110, endX: 400, endY: 110);
line3.setStroke(Color.RED);
line3.getStrokeDashArray().addAll( ...elements: 20d, 10d);
line3.setStrokeWidth(5);

root.getChildren().addAll(line1, line2, line3);
```



Line是一个非常轻量级的对象，可以在一个容器添加大量的实例而不用担心耗费太多的资源。

设定Line的起点和终点，可以很容易地移动这一线段控件。

Line的以上特点，使其非常适合于绘制各种图形或动态变化的图表。

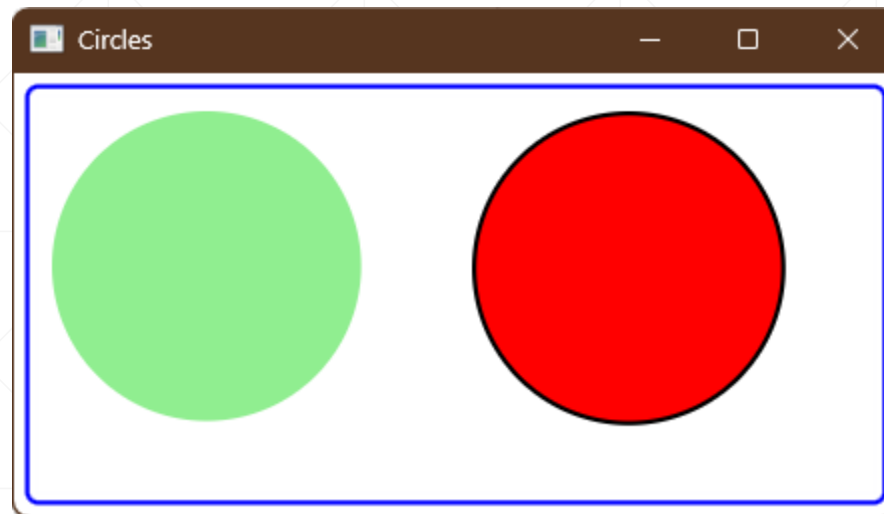
Circle: 代表一个圆

主要属性:

(centerX, centerY) 圆心

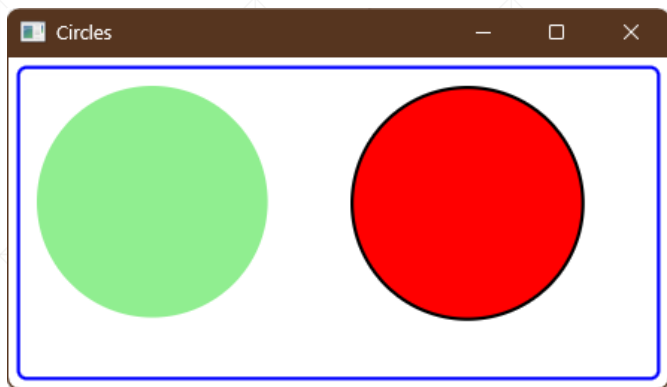
radius: 半径

可以设置线型和填充颜色。



CircleShapes.java

绘制圆的基本代码



CircleShapes.java

```
// centerX=0, centerY=0, radius=70, fill=LIGHTGRAY, stroke=null
Circle c1 = new Circle( centerX: 0, centerY: 0, radius: 70);
c1.setFill(Color.LIGHTGREEN);

// centerX=10, centerY=10, radius=70. fill=YELLOW, stroke=BLACK
Circle c2 = new Circle( centerX: 10, centerY: 10, radius: 70, Color.RED);
c2.setStroke(Color.BLACK);
c2.setStrokeWidth(2.0);
```

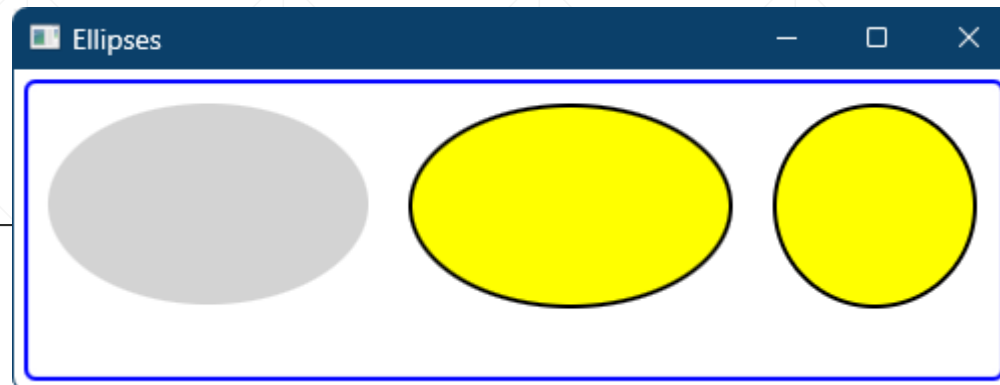
绘制椭圆

```
Ellipse e1 = new Ellipse( radiusX: 80, radiusY: 50);  
e1.setFill(Color.LIGHTGRAY);
```

```
Ellipse e2 = new Ellipse( radiusX: 80, radiusY: 50);  
e2.setFill(Color.YELLOW);  
e2.setStroke(Color.BLACK);  
e2.setStrokeWidth(2.0);
```

```
// Draw a circle using the Ellipse class (radiusX=radiusY=30)  
Ellipse e3 = new Ellipse( radiusX: 50, radiusY: 50);  
e3.setFill(Color.YELLOW);  
e3.setStroke(Color.BLACK);  
e3.setStrokeWidth(2.0);
```

EllipseShapes.java



示例中的三个椭圆，被放到了一个HBox中。

Ellipse类有以下重要属性：

- **centerX** – horizontal center point
- **centerY** – vertical center point
- **radiusX** – width of ellipse
- **radiusY** – height of ellipse

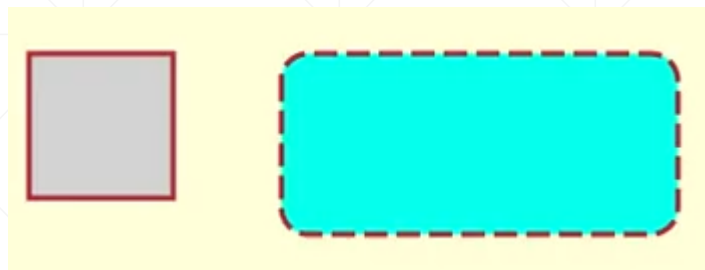
Rectangle: 代表一个矩形

重要属性:

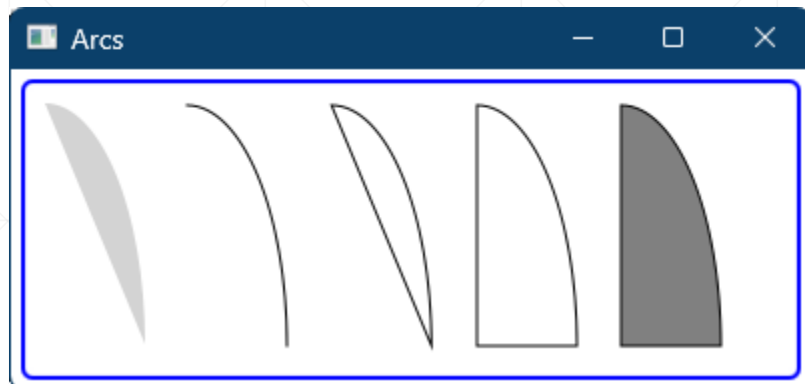
左上角坐标: (x,y)

宽 (width) 和高 (height)

通过设置它的arcWidth和arcHeight, 可以绘制圆角矩形。



绘制弧线与扇形



ArcShapes.java

- **centerX, centerY** – center point of arc
- **radiusX, radiusY** – width, height of arc
- **startAngle** – starting angle for arc
- **length** – length of arc
- **type** – closure specification

```
// An OPEN arc with a fill
Arc arc1 = new Arc( centerX: 0, centerY: 0, radiusX: 50, radiusY: 120,
    startAngle: 0, length: 90);
arc1.setFill(Color.LIGHTGRAY);
```

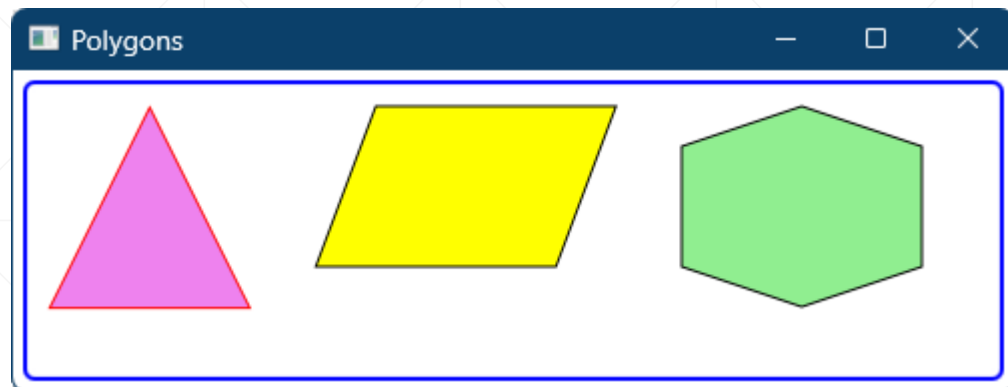
```
// An OPEN arc with no fill and a stroke
Arc arc2 = new Arc( centerX: 0, centerY: 0, radiusX: 50, radiusY: 120,
    startAngle: 0, length: 90);
arc2.setFill(Color.TRANSPARENT);
arc2.setStroke(Color.BLACK);
```

```
// A CHORD arc with no fill and a stroke
Arc arc3 = new Arc( centerX: 0, centerY: 0, radiusX: 50, radiusY: 120,
    startAngle: 0, length: 90);
arc3.setFill(Color.TRANSPARENT);
arc3.setStroke(Color.BLACK);
arc3.setType(ArcType.CHORD);
```

```
// A ROUND arc with no fill and a stroke
Arc arc4 = new Arc( centerX: 0, centerY: 0, radiusX: 50, radiusY: 120,
    startAngle: 0, length: 90);
arc4.setFill(Color.TRANSPARENT);
arc4.setStroke(Color.BLACK);
arc4.setType(ArcType.ROUND);
```

```
// A ROUND arc with a gray fill and a stroke
Arc arc5 = new Arc( centerX: 0, centerY: 0, radiusX: 50, radiusY: 120,
    startAngle: 0, length: 90);
arc5.setFill(Color.GRAY);
arc5.setStroke(Color.BLACK);
arc5.setType(ArcType.ROUND);
```

绘制多边形



PolygonShapes.java

要绘制多边形，需要计算出每个顶点的坐标，然后将所有顶点加入到Polygon对象的Points集合中，Polygon会自动两两连线绘制出多边形。

```
Polygon triangle1 = new Polygon();
triangle1.getPoints().addAll( ...elements: 50.0, 0.0,
                                   0.0, 100.0,
                                   100.0, 100.0);

triangle1.setFill(Color.VIOLET);
triangle1.setStroke(Color.RED);

Polygon parallelogram = new Polygon();
parallelogram.getPoints().addAll( ...elements: 30.0, 0.0,
                                           150.0, 0.0,
                                           120.00, 80.0,
                                           0.0, 80.0);

parallelogram.setFill(Color.YELLOW);
parallelogram.setStroke(Color.BLACK);

Polygon hexagon = new Polygon( ...points: 120, 0,
                                           180, 20,
                                           180, 80,
                                           120, 100,
                                           60, 80,
                                           60, 20);

hexagon.setFill(Color.LIGHTGREEN);
hexagon.setStroke(Color.BLACK);
```

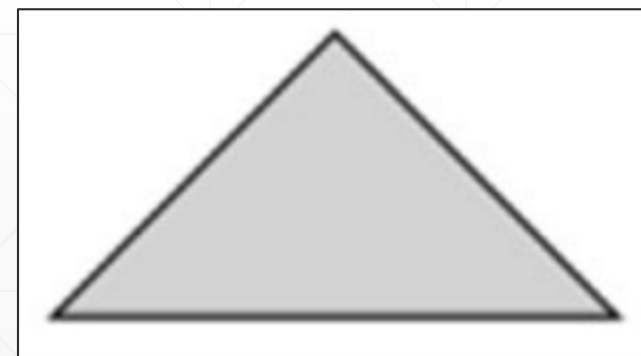
自己探索：绘制复杂图形



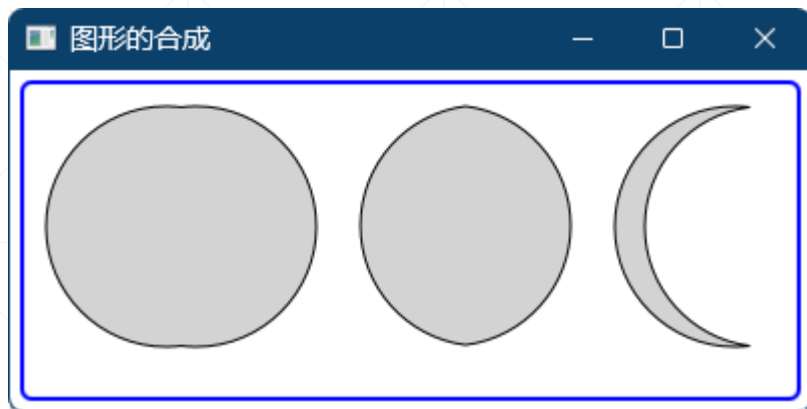
可以使用Path组合多个PathElement，绘制复杂的不规则图形。

特别地，可以使用SVGPath绘制SVG图形，网上可以找到大量的SVG图形

```
SVGPath svgTriangle = new SVGPath();  
svgTriangle.setContent("M50, 0 L0, 50 L100, 50 Z");  
svgTriangle.setFill(Color.LIGHTGRAY);  
svgTriangle.setStroke(Color.BLACK)
```



图形的“集合运算”



CombiningShapes.java

JavaFX的Shape，支持常规的集合运算：并集、交集和差集。

```
Circle c1 = new Circle(centerX: 0, centerY: 0, radius: 60);
Circle c2 = new Circle(centerX: 15, centerY: 0, radius: 60);

Shape union = Shape.union(c1, c2);
union.setStroke(Color.BLACK);
union.setFill(Color.LIGHTGRAY);

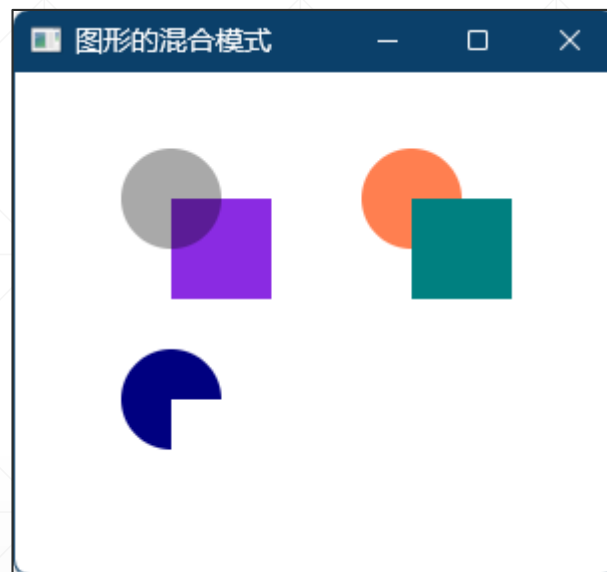
Shape intersection = Shape.intersect(c1, c2);
intersection.setStroke(Color.BLACK);
intersection.setFill(Color.LIGHTGRAY);

Shape subtraction = Shape.subtract(c1, c2);
subtraction.setStroke(Color.BLACK);
subtraction.setFill(Color.LIGHTGRAY);
```

图形混合模式

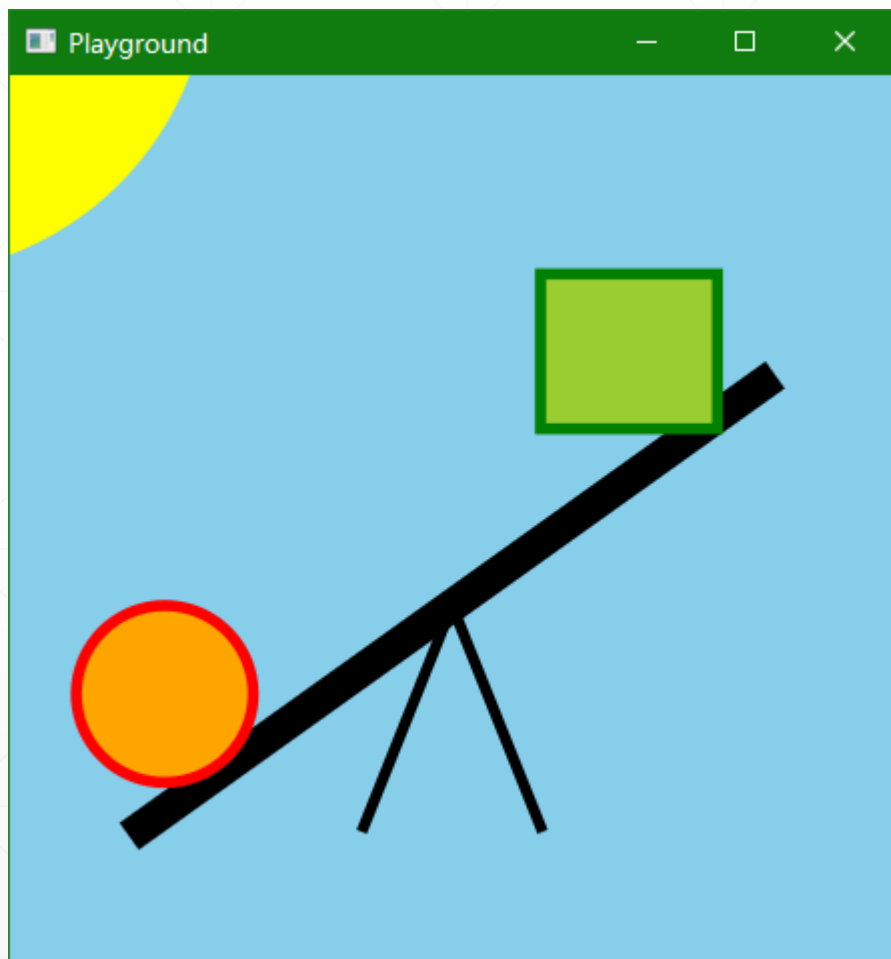
当两个图形重叠时，如何处理“重叠”的部分，是有讲究的，JavaFX称其为“混合模式（Blend Modes）”，有以下几个可选值：

- ADD
- MULTIPLY
- SRC_ATOP
- SRC_OVER
- SCREEN



BlenderShapes.java

基于基础图形绘制简单场景



playground.java

使用JavaFX所提供的Shape组件，
可以绘制一些简单的二维图形。

动态更改图形的属性



ChangingFace.java

这个示例使用基础图形绘制人脸，使用HBox排列两个按钮，使用VBox将按钮、人脸和Label控件全部组合起来。

仅仅只是修改了嘴的Arc的length属性，就实现了“变脸”

绘制数学图形



利用数学公式，使用基础图形对象，使用简单的循环、分支语句，往往可以绘制出很漂亮的图形，特别地，如果使用“分形几何”的思想，可以得到支持无级放大的几何图形。

```
Pane panel = new Pane();
for (int i = 0; i < 360; i++) {
    var e = new Ellipse(centerX: 150, centerY: 100, radiusX: 100, radiusY: 50);
    e.setStroke(Color.color(Math.random(), Math.random(), Math.random()));
    e.setFill(Color.WHITE);
    e.setRotate(i * 180 / 360);
    panel.getChildren().add(e);
}
```

BeautifulEllipse.java